

コンピュータアーキテクチャI

担当：佐々木 敬泰

第9回

プログラムの翻訳と起動

今日の内容

- プログラムの翻訳と起動
 - コンパイルの流れ
 - 逆アセンブル
 - ライブラリ
 - オブジェクトファイル
 - リンカとリロケーション
 - 静的ライブラリ
 - 動的ライブラリ

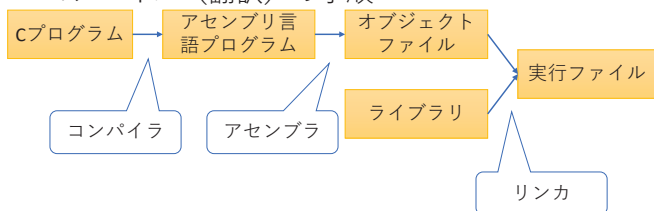
シラバス

1. イントロダクション
2. コンピュータの構成
3. 数の表現
4. 様々なプロセッサと命令セット
5. 基本命令
6. メモリアクセス命令
7. 分岐命令
8. 高級言語からアセンブラへの変換
9. プログラムの翻訳と起動
10. 加減算
11. 乗算アルゴリズムと乗算器
12. 除算アルゴリズムと除算器
13. 浮動小数点
14. 浮動小数点の乗算と誤差
15. 定期試験

プログラムの翻訳と起動

コンパイルの流れ

- コンパイラ
 - C/C++/Fortran/Go/D言語などから機械語へ変換
- コンパイル（翻訳）の手順



ライブラリを用いたプログラム

- ライブラリあり

```
int main(){
    char msg1[32]="Hello";
    char msg2[32]="World";
    /* 文字列を連結 */
    strcat(msg1, msg2);
    // 文字列を連結する
    // strcat()関数を呼ぶだけ
```

- ライブラリなし

```
int main(){
    char msg1[32]="Hello";
    char msg2[32]="World";
    char *p, *q;
    p=msg1;
    while(*p != '\0'){
        p++;
    }
    q=msg2;
    while(*q != '\0'){
        *q=*p;
        p++; /* まだまだ続くよ */
    }
}
```

ライブラリとは？

- プログラムは基本的に全部自分で書く
- とはいえ、
 - 入出力（キーボード、ディスプレイ、ファイル等）
 - ネットワーク通信
 - 数学関数（sin, cos, tan, ...）
 - 文字列処理等を全部自分で書くのは大変。
- よく使う関数は誰かが書いて共有
⇒ライブラリ

便利な関数の集まり

アセンブラ

- コンパイラはこれまで学んだ通り、C言語（高級言語）からアセンブリ言語に翻訳するツール
- アセンブラは、アセンブリ言語をオブジェクトファイルに変換するツール
 - アセンブリ言語
 - テキストファイル
 - 機械語と一対一対応する
 - 疑似命令やラベルを含む
 - オブジェクトファイル
 - 機械語を含むバイナリファイル
 - 解決できないラベル（関数や大域変数）等はテーブルに記録（アセンブルの時点では未定）

グロッサリー（用語整理）

ソースプログラム

人間の書くプログラム。通常テキストファイル。

コンパイル（翻訳）

ソースファイルをアセンブラに翻訳する作業

コンパイラ

コンパイルするツール。gccやclang、javac等。

アセンブリ言語（アセンブル）

基本的に機械語と1対1対応した人間が直接読み書きできるプログラム。変換作業はアセンブルと呼ぶ。

アセンブラ

アセンブリ言語から機械語へ変換するツール。

オブジェクトファイル（目的ファイル）

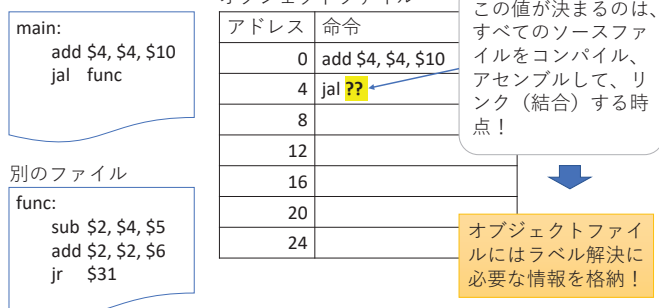
アセンブラの生成する中間ファイル。機械語とリンクのためのリロケーション情報等を含む。

実行ファイル/実行可能ファイル

実行可能なファイル。静的リンクの場合は実行に必要な情報を全て含む。

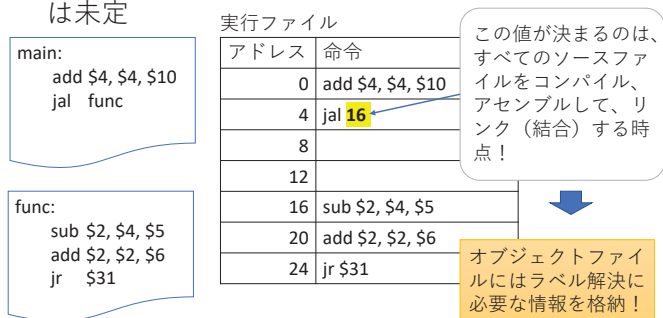
ラベル解決

- アセンブル時点では、参照ファイル（ライブラリや他のアセンブラプログラム）のラベル位置は未定



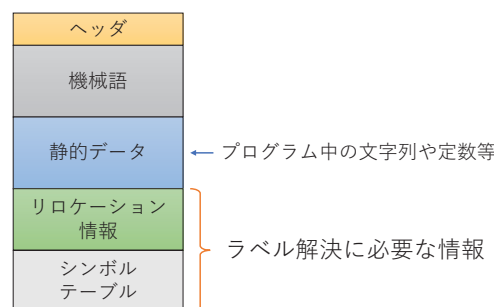
ラベル解決

- アセンブル時点では、参照ファイル（ライブラリや他のアセンブラプログラム）のラベル位置は未定



オブジェクトファイル

- 機械語＋静的データ＋未解決の参照情報
- 他にもデバッグ情報等も含む



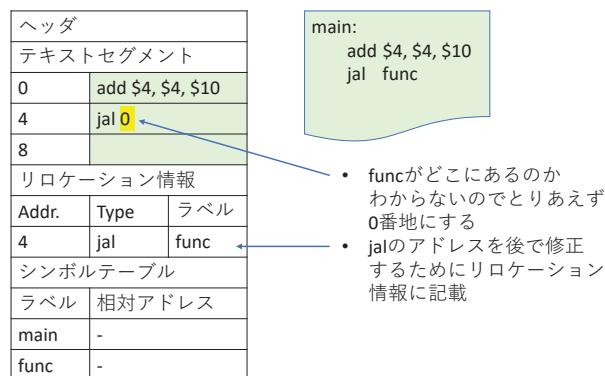
実行可能ファイルの構造

- オブジェクトファイルや実行可能ファイルは中身によって領域（セグメント）に分かれている

ヘッダ	ファイルサイズ、各セグメントのサイズや開始位置、プログラムの開始位置等
テキスト	機械語
データ(RO)	読み出し専用のデータ（定数）
データ(RW)	読み書き可能なデータ（初期値）
リロケーション情報	アドレスやタイプ等
シンボル情報	ラベルや変数名等
(他、いろいろ)	デバッグ用の情報等

WindowsならPE(Portable Executable)、UNIX系ならELF(Executable and Linkable Format)という規格を採用したものが多く。

リンク（mainのアセンブル）



リロケーション情報のType

- 未解決のラベルはリンク時に定数に置き換える
- 当てはめるデータは命令依存

jal func

Type Jの26bit



lw \$4, idx

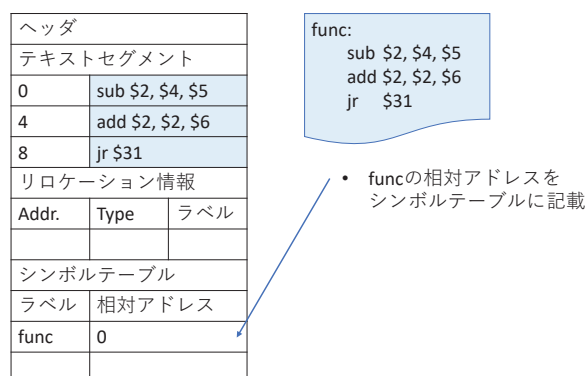
\$gpからの相対アドレス



他にもラベルの32bit絶対アドレス等

リロケーション情報		
Addr.	Type	ラベル
4	jal	func
10	lw	idx
:	:	:

リンク（funcのアセンブル）



リンク（実行ファイル作成）

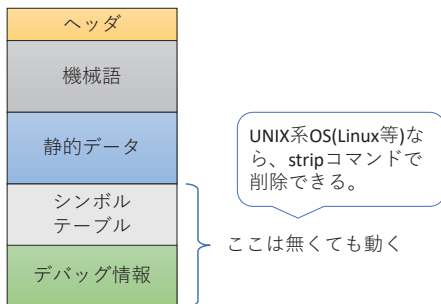
ヘッダ			実行ファイル		
テキストセグメント			テキストセグメント		
0	add \$4, \$4, \$10		0	sub \$2, \$4, \$5	
4	jal 0		4	add \$2, \$2, \$6	
8			8	jr \$31	
リロケーション情報			リロケーション情報		
Addr.	Type	ラベル	Addr.	Type	ラベル
4	jal	func			
シンボルテーブル			シンボルテーブル		
ラベル	相対アドレス		ラベル	相対アドレス	
main	-		func	0	
func	-				

コンピュータアーキテクチャI

17

実行ファイル

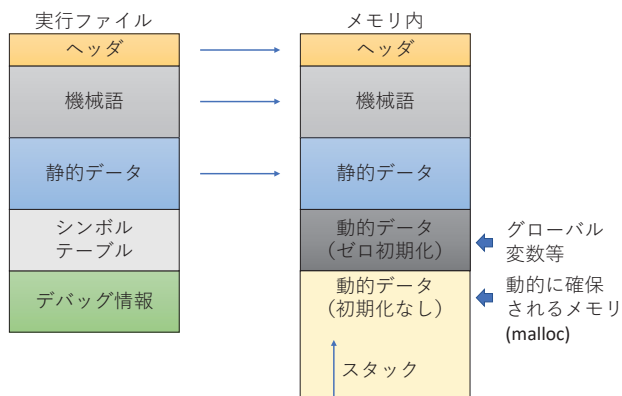
- リンク後に生成される実行には実行に必要な機械語はすべて含まれる（自分の書いたコード＋ライブラリ）



コンピュータアーキテクチャI

19

メモリーイメージ

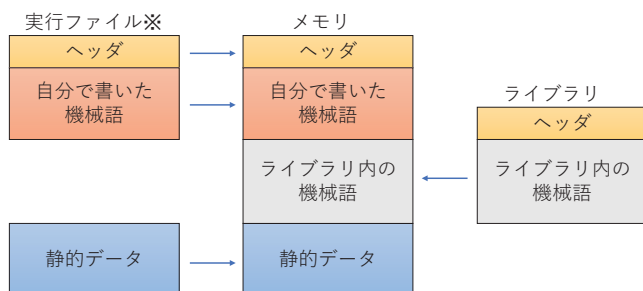


コンピュータアーキテクチャI

21

動的リンクライブラリ

- リンク時にライブラリを結合せず、実行時にメモリ上でライブラリを読み込みリンクする



※ 実行ファイルやライブラリには、動的リンクに必要な情報（リロケーション情報やシンボルテーブル等）も含む

コンピュータアーキテクチャI

23

実際のオブジェクトファイルの例

オブジェクトファイル（抜粋）

```

14: 8f990000 lw $t9, 0($gp)
18: 3c040000 lui $a0, 0x0
1c: 24840000 addiu $a0, $a0, 0
20: 0320f809 jalr $t9

```

```

int main(){
    printf("Hello world\n");
    return 0;
}

```



printf関数の場所
文字列データ"Hello world\n"の
置いてある場所

```

4003b4: 8f998038 lw $t9, -32712(gp)
4003b8: 3c040047 lui $a0, 0x47
4003bc: 24840104 addiu $a0, $a0, 260
4003c0: 0320f809 jalr $t9

```

コンピュータアーキテクチャI

18

ロード

- 実行ファイルを実行する時の流れ

1. 実行ファイルのヘッダを読んで、必要なメモリを確保する
 - 機械語や静的データだけでなく、スタック領域等の領域も確保する
2. 実行ファイルからテキストセグメントと静的データセグメントをメモリ上にコピーする
3. 実行時に引数があればスタック上にコピーする
4. レジスタを初期化する
5. スタートアップルーチンにジャンプする

コンピュータアーキテクチャI

20

動的リンク

- 実行ファイルには、必要なデータはすべて入っている（自分の書いたコード＋ライブラリ）
- しかし、
 - ライブラリはよく使う関数の集合なので、各々の実行ファイルに含めると冗長
 - ライブラリを更新したときに、すべてリンクし直す必要がある



動的リンクライブラリ



コンピュータアーキテクチャI

22

コンパイラとインタプリタ

- コンパイラ

- ソースプログラムから機械語に翻訳する
- 実行ファイルは特定のCPUやOS上でしか動かない
- 実行ファイル単体で実行できる（動的リンクの場合は動的ライブラリは必要）

例）C, C++, D, Objective-C, Fortran等が多い

- インタプリタ

- ソースプログラムを順次解釈しながら実行
- プログラムの実行には専用のプログラム（インタプリタ）が必要

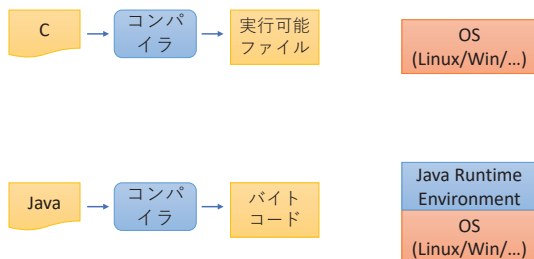
例）python, perl, ruby

⇒Cのインタプリタやperlのコンパイラも存在

コンピュータアーキテクチャI

24

- Javaはコンパイルが必要だが、実行にはランタイム(Java Runtime Environment)が必要



- Javaはコンパイルが必要だが、実行には別のランタイムが必要。何故？
 - Cコンパイラ等はターゲットマシン以外の環境では動かない。
 - Windows用のバイナリはLinuxでは動かない
 - 同じLinux環境でもMIPS用のバイナリはx64マシンでは動かない
- JavaはWrite once, run anywhere（一度書けばどこでも実行できる）を目指している
⇒Javaのバイトコードはマシン非依存
ただし、ランタイムは依存

JavaとJIT(Just-In-Time)コンパイラ

- Javaのバイトコードはコンパイル済みのコード
 - ソースコードから字句解析や構文解釈、コード割当てはする処理が不要なのでそこそ早い
 - 最適化は不十分
- そこで、高速化のために実行時にネイティブコード（機械語）に翻訳する技術JIT(Just-In-Time)コンパイラが用いられている

